PROP: Physically Based Rendering of Optical Phenomena

Daniel Mayer Cecilia Halbritter Lukas Walker Kai Stampa

RheinMain University of Applied Sciences, Wiesbaden, Germany

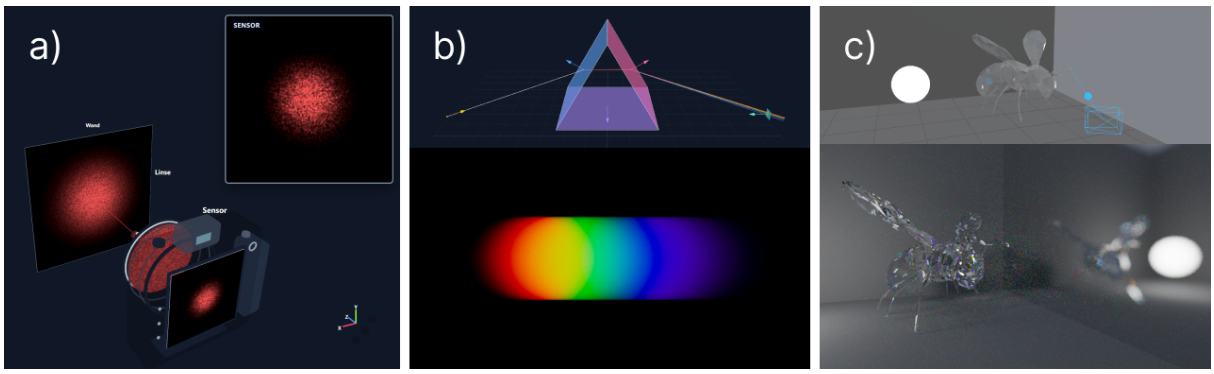


Figure 1: *Overview of the interactive PROP application: (a) browser-based FFT laser-speckle visualization with wall, lens, and sensor planes, (b) prism-scene preview and spectral dispersion produced by the 2D-SLT, and (c) scene preview and rendered output of the CUDA-SPT. The standalone headless C++ laser-speckle application used for the experiments is not shown.*

Abstract

Prism dispersion and laser speckle require different light representations: wavelength-resolved transport for dispersion and phase-preserving wave propagation for speckle. This work examines which properties of light must be retained for these phenomena and the computational and numerical trade-offs of problem-specific rendering models. PROP includes a GPU-parallel 2D Spectral Light Tracer for projected prism spectra, an existing CUDA Spectral Path Tracer integrated for camera-based spectral rendering, and a standalone headless C++ laser-speckle application that implements direct Rayleigh–Sommerfeld and FFT-based angular-spectrum propagation in the same optical pipeline. For interactive visualization, the FFT path was separately reimplemented in WebGPU and integrated with the two spectral renderers in a Vite-based frontend.

The prism benchmarks characterize runtime scaling rather than rank the renderers directly because they compute different observables and use different sampling configurations. The 2D Spectral Light Tracer scales efficiently with the number of emitted light samples but is limited by its CPU-based spectral-to-RGB conversion; the CUDA Spectral Path Tracer runtime more closely follows the number of traced spectral samples. Across 14 deterministic configurations in the headless C++ application, FFT-based propagation reduced the mean runtime from 302.5 s to 0.991 s, a speedup of 305.2×, while achieving a mean detector-image correlation of 0.929 with the direct integrator. Agreement was highest under normal incidence and decreased for increasingly off-axis geometries. The results show that matching the light representation and renderer scope to the target phenomenon can substantially reduce computational cost, although the rendered observable and numerical agreement remain tied to explicit model and geometry assumptions.

1 Introduction

Physically based light simulation remains a central challenge in computer graphics because rendering paradigms retain different properties of the light field. Rasterization prioritizes efficiency for real-time applications, whereas ray-based methods model geometric light transport and naturally support effects such as reflections, refractions, shadows, and global illumination [PJH23]. However, geometric light transport based only on three RGB channels does not retain the wavelength or phase information required to model phenomena such as dispersion, diffraction, and interference [RBA09, IWR*14, Goo07].

This work focuses on two representative phenomena with different physical requirements. Prism dispersion separates light into wavelength-dependent components because the refractive index of an optical material varies with wavelength [Zim23]. Laser speckle arises from the coherent superposition of contributions scattered by an optically rough surface and therefore depends on both amplitude and phase [Goo07]. Consequently, prism dispersion requires

wavelength-resolved geometric transport, whereas laser speckle additionally requires a phase-preserving wave-optical model [SY21].

PROP investigates how problem-specific renderers can reproduce these phenomena while avoiding the complexity of a general-purpose rendering system. Rather than supporting arbitrary scenes and optical effects, each renderer is restricted to the components required for its target phenomenon. The project addresses two questions: which properties of light must be retained to model prism dispersion and laser speckle, and which computational and numerical trade-offs arise from the corresponding problem-specific models?

The main contributions of this work are:

- a GPU-parallel 2D Spectral Light Tracer (2D-SLT) that simulates wavelength-dependent forward light transport through a prism and records the resulting spectrum on a configurable image plane;
- the integration and analysis of an existing CUDA Spectral Path Tracer (CUDA-SPT) [Bau26], enabling a qualitative comparison between projected spectral distributions and camera-based spectral rendering;
- a standalone headless C++ laser-speckle application that implements both direct Rayleigh–Sommerfeld integration and FFT-based angular-spectrum propagation within the same optical pipeline; and
- a Vite-based interactive application that integrates the two spectral renderers and a separate WebGPU reimplementation of the FFT-based laser-speckle path through a consistent frontend.

The prism renderers are evaluated for runtime scaling and analyzed qualitatively because they compute different observables. The two laser-speckle propagation schemes are compared in the same standalone headless C++ application using runtime, relative error, and detector-image correlation; the browser implementation is used only for interactive visualization and is not evaluated. These evaluations characterize the computational behavior of the problem-specific implementations and show where their model assumptions limit numerical agreement or the observable that can be rendered.

2 Related Work

Because prism dispersion and laser speckle rely on different rendering paradigms, this section discusses spectral and wave-based rendering separately.

2.1 Spectral Rendering

In a recent survey, Kim and Gotchev [KG24] describe the complete spectral rendering pipeline, from spectral input to RGB output. They discuss spectrum representations, reconstruction from RGB colors, spectral ray-tracing algorithms, and conversion of rendered spectra back to RGB. Spectra can be represented pointwise as wavelength–reflectance or wavelength–radiance pairs, or approximated with basis functions [KG24]. In spectral ray tracing, replacing transported RGB information with a single wavelength per ray leads to increased color noise. To address this, Evans and McCool [EM99] introduced stratified wavelength clusters, which track multiple wavelengths on the same ray. This concept was later extended by hero wavelength sampling [WND*14], where only one hero wavelength is sampled randomly while the remaining wavelengths are placed at fixed intervals. Because the wavelengths still share one geometric path, wavelength-dependent refraction requires dropping all but the primary wavelength (degradation), spawning one ray per wavelength (splitting), or evaluating the remaining wavelengths retrospectively (deferral) [EM99].

PBRT-v4 [PJH23] combines these concepts by using hero wavelength sampling together with degradation at dispersive interfaces. Other spectral renderers either implement wavelength-dependent dispersion [Bau26] or omit it entirely [WSND26].

2.2 Wave-based Rendering

Laser speckle results from the coherent superposition of randomly phased contributions scattered by an optically rough surface [Goo07]. Numerical simulations therefore represent light as a complex-valued field to preserve amplitude and phase. Free-space propagation can be evaluated using the Rayleigh–Sommerfeld integral or the angular-spectrum method. Direct integration provides flexible point-to-point propagation at high computational cost, while FFT-based methods operate efficiently in the spatial-frequency domain [Goo05, SW06].

Wave-optical effects have also been integrated into rendering systems. Steinberg and Yan render subjective speckle from statistically described rough surfaces without explicitly resolving their microscopic geometry [SY22]. Steinberg et al. present a more general physical-optics renderer that transports coherence, spectral, and polarization information through complex scenes using wave BSDFs [SSY22]. More recently, Steinberg and Pharr extend the path-integral formulation of light transport to wave optics, explicitly modeling interference between propagation paths [SP25].

These approaches target general scenes and require correspondingly complex transport and sampling models. Our renderer instead uses a controlled setup comprising a rough wall, a thin lens, and a sensor, enabling a transparent implementation and direct comparison of Rayleigh–Sommerfeld and angular-spectrum propagation.

3 Method

This section describes the renderer implementations and their physical and numerical models.

Implementation Scope PROP combines project-specific implementations with an existing spectral renderer. The 2D-SLT and a standalone headless C++ laser-speckle application were developed for the project. The C++ application contains both the direct Rayleigh–Sommerfeld and FFT-based angular-spectrum backends and formed the basis for algorithm development and all laser-speckle experiments. The existing CUDA-SPT by Baudoin [Bau26] was integrated into PROP as a camera-based spectral comparison point; its path-tracing implementation is not claimed as an original contribution. For interactive presentation, a separate Vite-based frontend and renderer-specific integration modules were developed. The FFT-based laser-speckle path was reimplemented in WebGPU for this frontend, following the C++ optical model without connecting to or invoking the headless application.

3.1 Prism Dispersion

A prism separates light because refraction depends on wavelength. When a ray enters or leaves the glass, its direction can change. The material's refractive index determines this deflection and varies with wavelength. Shorter wavelengths such as violet are therefore generally deflected more strongly than longer wavelengths such as red, causing the components of broadband light to leave the prism in different directions [Zim23, Bal08].

An RGB-only transport representation does not resolve the wavelength dependence required to compute prism dispersion directly. Both prism renderers therefore sample individual wavelengths and evaluate the source and prism material at the sampled wavelength. Both support source spectra, wavelength-dependent refraction, interface reflection, and absorption, although the models and use of absorption differ between implementations.

3.1.1 Spectral Optical Model

The description of the optical model follows the order of light propagation: the source spectrum determines which wavelengths are present, refraction changes their directions, interface reflection selects reflected or transmitted paths, and absorption attenuates light inside the glass.

Spectral Power Distribution The spectral power distribution (SPD) describes how a light source's power is distributed across wavelengths. It is represented by $E(\lambda)$ over the visible range (380–780 nm): a larger value of $E(\lambda)$ means that the source emits more power near that wavelength [KDR18]. Radiometrically, $S(\lambda)$ is measured in $\mathrm{W\,nm^{-1}}$; integrating it over the visible range gives the total emitted power:

$$\Phi_e = \int_{380\,\mathrm{nm}}^{780\,\mathrm{nm}} S(\lambda)\,\mathrm{d}\lambda. \tag{1}$$

Different light sources exhibit characteristic spectral distributions, ranging from smooth curves to narrow peaks (Fig. 2).

Refraction Refraction occurs whenever a ray crosses a boundary between transparent media, such as air and glass. The wavelength-dependent refractive index $n(\lambda)$ determines the ray deflection at wavelength λ. Because $n(\lambda)$ varies with wavelength, the same prism produces slightly different paths for different parts of the spectrum.

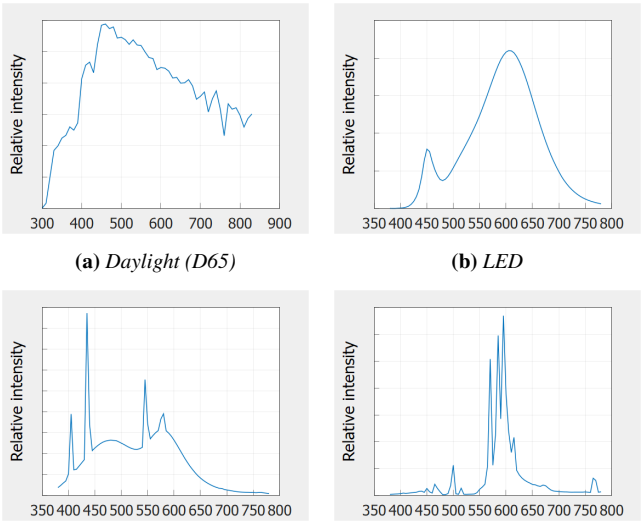
The 2D-SLT evaluates $n(\lambda)$ using the Sellmeier equation [Hec16]:

$$n^2(\lambda) = 1 + \frac{B_1\lambda^2}{\lambda^2 - C_1} + \frac{B_2\lambda^2}{\lambda^2 - C_2} + \frac{B_3\lambda^2}{\lambda^2 - C_3}, \tag{2}$$

where λ is the wavelength and B_i and C_i are material-specific Sellmeier coefficients. The CUDA-SPT instead uses the empirical Cauchy approximation

$$n(\lambda) = A + \frac{B}{\lambda^2} + \frac{C}{\lambda^4}, \tag{3}$$

where A, B, and C are material-specific coefficients. This approximation is suitable over a limited wavelength interval such as the visible spectrum [Hec16].



(a) *Daylight (D65)* **(b)** *LED*

(c) *Fluorescent lamp* **(d)** *High-pressure sodium lamp*

Figure 2: *Relative spectral power distributions of different light sources based on CIE datasets [noa].*

Both renderers then use Snell's law to compute the refracted direction:

$$n_i \sin(\theta_i) = n_t \sin(\theta_t), \tag{4}$$

where n_i and n_t are the wavelength-dependent refractive indices before and after the boundary, and θ_i and θ_t are the incident and transmitted angles measured from the surface normal. Thus, the material model supplies the indices and Snell's law converts them into a direction change.

Interface Reflection A transparent boundary does not transmit all incident light. A fraction R is reflected, while the remaining fraction is transmitted. The exact reflected fraction depends on the refractive indices and the angle at which the ray reaches the boundary. For the two polarization components, the Fresnel reflectances are [Pas13]

$$R_\perp = \left| \frac{n_i\cos(\theta_i) - n_t\cos(\theta_t)}{n_i\cos(\theta_i) + n_t\cos(\theta_t)} \right|^2, \tag{5}$$

$$R_\parallel = \left| \frac{n_t\cos(\theta_i) - n_i\cos(\theta_t)}{n_t\cos(\theta_i) + n_i\cos(\theta_t)} \right|^2. \tag{6}$$

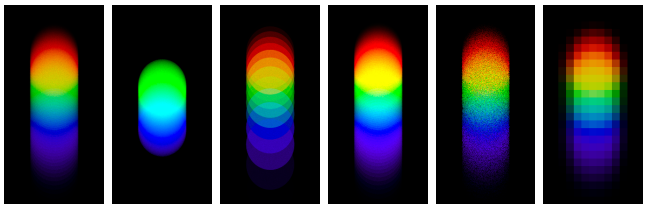
Unpolarized light contains both components, so the 2D-SLT uses their average [Hec16]:

$$R = \frac{R_\perp + R_\parallel}{2}. \tag{7}$$

The CUDA-SPT uses the computationally simpler Schlick approximation [Maj21],

$$R \approx R_0 + (1 - R_0)(1 - \cos\theta_i)^5, \qquad R_0 = \left(\frac{n_i - n_t}{n_i + n_t}\right)^2. \tag{8}$$

In both renderers, the computed reflectance is used to select reflection with probability R and refraction with probability $1 - R$.



(a) **(b)** **(c)** **(d)** **(e)** **(f)**

Figure 3: *Effects of different render settings on the resulting image. (a) Reference image. (b) Wavelength range limited to* 450–550 nm*. (c) Spectral resolution reduced to 16 wavelength steps. (d) Under the current linear accumulation and exposure model, doubling either the light power or image exposure produces the same displayed result. (e) Reducing the number of light samples to 2% of the reference count introduces color noise. (f) Reduced image resolution.*

Absorption Even a transmitted ray can lose intensity while traveling through the prism. This wavelength-dependent attenuation over an optical path of length d is modeled with the Beer–Lambert law [MPP20]:

$$T(\lambda) = \exp(-\sigma_a(\lambda)\, d)\,, \tag{9}$$

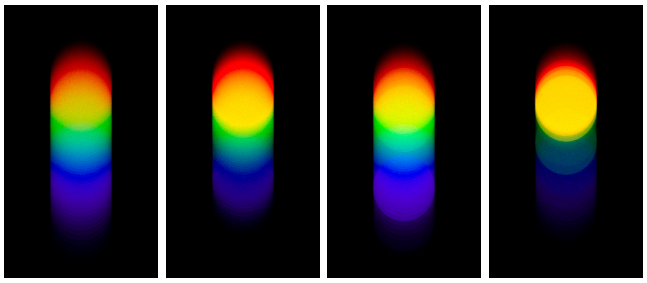
where $\sigma_a(\lambda)$ is the absorption coefficient of the material at wavelength λ. Longer paths and larger absorption coefficients reduce transmission. The remaining intensity is $I' = IT(\lambda)$. Both prism renderers use this law, either for reduction of ray intensity or probabilistic path termination.

3.1.2 2D-SLT

The 2D-SLT is a forward Monte Carlo light tracer [DLW93] implemented in C++. Here, *forward* means that rays start at the light source and travel through the prism toward a configurable two-dimensional image plane. The renderer records where light reaches this plane rather than producing a camera view. Each ray stores an origin, direction, wavelength, and relative intensity and is propagated until it reaches the image plane, is absorbed, or exceeds the recursion limit. Reflection and absorption are sampled probabilistically, while an explicit random seed makes repeated runs reproducible.

Scene and Configuration The scene contains three main components: an area light, a triangular prism, and an image plane that acts as the receiver. The area light can be square or circular; its position, orientation, size, and SPD define the ray origins and wavelengths. The prism can be transformed and assigned an apex angle and optical material. The image plane can be positioned, oriented, and resized to select where transmitted light is recorded. Additional render settings control recursion depth, wavelength range, spectral resolution, emitted power, light-sample count, image exposure, and output resolution (Fig. 3).

SPD Preprocessing In the 2D-SLT, the spectral power distribution is represented by a discrete set of uniformly spaced wavelength samples. The SPD data are loaded from CSV files containing CIE reference data in the form of wavelength–power pairs [noa].



(a) *Daylight (D65)* **(b)** *LED* **(c)** *Fluorescent lamp* **(d)** *High-pressure sodium lamp*

Figure 4: *Effects of different spectral power distributions on the resulting image. The corresponding SPDs are plotted in Fig. 2.*

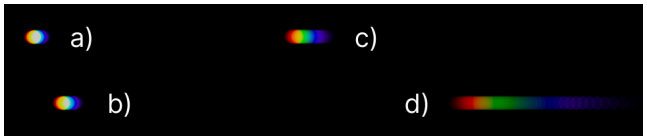


Figure 5: *Dispersion patterns produced by different prism materials on the same image plane. For the tested materials, stronger wavelength-dependent refraction produces a larger spatial separation of wavelengths and shifts the projected pattern. (a) N-FK51A. (b) N-BK7. (c) N-LAK34. (d) N-SF57.*

For each spectral sample, the renderer linearly interpolates the power between the two nearest input entries. Increasing the number of spectral samples improves spectral resolution and produces smoother color transitions (compare Fig. 3a with Fig. 3c). The resampled spectral power distribution is subsequently converted into a cumulative distribution function (CDF) for stochastic wavelength sampling. Wavelengths with greater spectral power are therefore more likely to be selected. Fig. 4 depicts the rendered rainbow patterns of different SPDs.

Material Preprocessing The relevant optical properties of the selected prism material are the wavelength-dependent absorption coefficient $\sigma_a(\lambda)$ and the Sellmeier coefficients B_i and C_i. These parameters are loaded from two CSV files generated from optical glass data sheets provided by Schott [SCH21]. The resulting material-dependent patterns are shown in Fig. 5.

Stochastic Ray Sampling Each ray stores an origin, propagation direction, wavelength, and intensity. The ray origin is sampled randomly over the area light, while its propagation direction is determined by the orientation of the light source. The wavelength is sampled according to the CDF derived from the spectral power distribution. Each ray is assigned the same initial intensity, computed as the total emitted light power divided by the number of sampled rays. The ray intensity is a relative, unitless quantity rather than an absolute radiometric measurement.

Recursive Light Tracing Before tracing begins, the renderer evaluates the refractive index of the selected prism material at the wavelength of each ray using the Sellmeier equation (Eq. 2). The

rays are then processed independently and in parallel up to the configured recursion depth.

At every step, the renderer first finds the closest intersection with either the image plane or a prism surface. Reaching the image plane records a contribution and ends the path. At a prism surface, the renderer accounts for absorption and samples either reflection or refraction:

1. If the ray exits the prism, the transmission through the traversed path is computed using the Beer–Lambert law (Eq. 9). The ray is then terminated with probability $1 - T(\lambda)$ to account for absorption.
2. The Fresnel reflectance is computed using the unpolarized Fresnel equations (Eq. 7). The ray is then stochastically reflected or refracted according to the computed reflectance.

This procedure repeats until the ray reaches the image plane or the maximum recursion depth. The renderer assumes perfectly smooth prism surfaces and therefore does not simulate diffuse surface scattering.

Resulting Image When a ray reaches the image plane, its intersection point is mapped to a pixel and its intensity is accumulated in the wavelength bin carried by that ray. The intermediate image is therefore spectral: each pixel stores contributions at several wavelengths rather than an RGB triplet. This spectral information is converted to displayable color only after all rays have been traced.

The accumulated spectral intensities $S(\lambda_i)$ are first converted to CIE XYZ tristimulus values using an analytic approximation of the CIE 1931 color matching functions $\bar{x}$, $\bar{y}$, and $\bar{z}$ [WSS13]:

$$\begin{aligned} X &= \sum_i S(\lambda_i)\,\bar{x}(\lambda_i), \\ Y &= \sum_i S(\lambda_i)\,\bar{y}(\lambda_i), \\ Z &= \sum_i S(\lambda_i)\,\bar{z}(\lambda_i). \end{aligned} \tag{10}$$

The sums measure how strongly the spectrum stimulates the CIE X, Y, and Z color-matching channels. The resulting XYZ values are transformed to linear sRGB using the standard XYZ-to-sRGB conversion matrix defined for the sRGB color space with a D65 reference white (Fig. 2a) [IEC99]. Finally, the linear RGB values are scaled by the user-defined exposure and converted to gamma-corrected sRGB using the standard sRGB transfer function before being written to the output PPM image [IEC99]. Thus, the simulation retains wavelength-dependent transport and reduces it to RGB only for display.

Parallelization and Complexity The computational work of the 2D-SLT is

$$\mathcal{O}(L \cdot D + W \cdot H \cdot S), \tag{11}$$

where L denotes the number of emitted light samples and D the maximum recursion depth. The first term, $\mathcal{O}(L \cdot D)$, corresponds to the recursive light tracing stage. Since individual rays are processed independently, this stage is parallelized on the GPU using CUDA. The second term, $\mathcal{O}(W \cdot H \cdot S)$, corresponds to the conversion of the accumulated spectral image to RGB, where $W \cdot H$ denotes the number of image pixels and S the number of spectral samples. Because pixels are independent, this stage is parallelized on the CPU with OpenMP [DM98].

3.1.3 CUDA-SPT

To complement the project-specific 2D-SLT, we integrated the existing C++ CUDA-based spectral path tracer by Theo Baudoin [Bau26]. It provides a camera-based spectral comparison point for the application and qualitative analysis. Unless noted otherwise, this subsection describes the existing CUDA-SPT; our contribution is its integration, scene setup, and evaluation within PROP.

Like the 2D-SLT, the CUDA renderer traces one wavelength per ray, evaluates the prism refractive index as $n(\lambda)$, and converts accumulated spectral contributions through CIE XYZ to sRGB. The central difference is the direction of transport. The 2D-SLT starts at the light and records where rays reach a receiver. The CUDA renderer starts rays at the camera and follows paths through the scene to estimate the light reaching each pixel. It also supports a broader range of three-dimensional geometry and materials.

Camera and Wavelength Sampling Each CUDA thread computes one image pixel. For every pixel, the renderer generates several camera paths and samples wavelengths over the configurable default range from 380 nm to 780 nm. Two random values jitter each camera ray within the pixel footprint, avoiding repeated evaluation at the pixel center.

The wavelength range is sampled using stratified Monte Carlo integration as introduced by Evans and McCool [EM99]. In practical terms, the renderer divides the visible interval into N_λ equally sized bins and selects one random wavelength inside each bin. This covers the visible range more evenly than independent wavelength sampling. For every pixel sample p and spectral bin i, the random value $\mathrm{rand}_{p,i} \in [0,1)$ selects the exact wavelength inside that bin:

$$\lambda_{p,i} = \lambda_{\min} + (i + \mathrm{rand}_{p,i})\,\Delta\lambda, \qquad \Delta\lambda = \frac{\lambda_{\max} - \lambda_{\min}}{N_\lambda}. \tag{12}$$

Unlike the 2D-SLT, the CUDA renderer does not use the SPD to decide which wavelengths are selected. Instead, it samples all wavelength bins and then weights the contribution of a sampled wavelength by the source and material values. For an emissive material, for example, the contribution is $E(\lambda_{p,i})$: wavelengths with high spectral power contribute more radiance, whereas those with little power contribute less and those with none contribute nothing.

Spectral Accumulation For each sampled wavelength, the renderer traces a camera path and estimates the spectral radiance reaching the camera. Before conversion to sRGB, the accumulated CIE XYZ value is

$$\mathbf{X}_{\mathrm{pixel}} = \frac{1}{N_p} \sum_{p=1}^{N_p} \sum_{i=1}^{N_\lambda} \frac{L_{p,i}(\lambda_{p,i})\,\bar{\mathbf{c}}(\lambda_{p,i})\,\Delta\lambda}{K_Y}, \tag{13}$$

where $\mathbf{X}_{\mathrm{pixel}} = (X,Y,Z)$, N_p is the number of pixel samples, N_λ is the number of spectral samples, and $\lambda_{p,i}$ is the jittered wavelength used for spectral sample i of pixel sample p. The term $L_{p,i}(\lambda_{p,i})$ denotes the scalar spectral radiance returned by the path tracer for that camera path at the sampled wavelength. Furthermore, $\bar{\mathbf{c}}(\lambda) = (\bar{x}(\lambda), \bar{y}(\lambda), \bar{z}(\lambda))$ denotes the CIE 1931 color matching functions, $\Delta\lambda = (\lambda_{\max} - \lambda_{\min})/N_\lambda$ is the width of one spectral

stratum, and K_Y is the normalization constant corresponding to the integral of the CIE $\bar{y}$ function over the sampled wavelength range. Conceptually, the equation averages camera-path samples, weights each radiance value by the color-matching functions at its wavelength, and accounts for the wavelength-bin width. The accumulated XYZ values are then transformed to linear sRGB and written to the image after tone mapping and gamma correction.

Scene and Material Scope The material model is broader than in the 2D-SLT. Besides dispersive dielectrics, the CUDA renderer supports lambertian, metallic, emissive, and spectral materials, as well as 3D primitives such as spheres, rectangles, triangles, boxes, and triangular prisms. At diffuse or spectral surfaces, new directions are sampled using cosine-weighted Monte Carlo sampling. Emissive materials contribute wavelength-dependent radiance, and direct lighting from emissive objects is estimated explicitly and combined with path sampling using a power heuristic. This allows the renderer to handle more general three-dimensional scenes, including indirect illumination and visible objects viewed through or around the prism.

For dispersive dielectrics, the CUDA renderer evaluates the refractive index with the Cauchy approximation (Eq. 3), applies Snell's law (Eq. 4), and stochastically samples reflection or refraction using Schlick's approximation (Eq. 8). Total internal reflection forces reflection, and optional absorption attenuates the scalar path throughput according to the Beer–Lambert law (Eq. 9). The path throughput remains scalar because each ray represents one wavelength. A visible color emerges only after contributions from many wavelengths have been combined through the CIE color-matching functions.

Computational Complexity The dominant kernel cost can be approximated by

$$\mathcal{O}(WHN_pN_\lambda DM(1+E)), \tag{14}$$

where WH is the number of pixels, N_p the pixel-sample count, N_λ the spectral-sample count, D the maximum path depth, M the number of scene objects tested for intersection, and E the number of emissive objects considered during direct-light estimation. This implementation-specific cost model is approximate because paths may terminate early and execute material-dependent code. The current implementation has no spatial acceleration structure, so each ray may have to test all scene objects. Its intersection cost therefore grows linearly with scene complexity.

Observation Model The two prism renderers do not produce the same observable, even with similar prism and light configurations. This distinction is essential when comparing their color patterns.

The 2D-SLT starts rays at the light source and records where they reach an image plane. Its result can be interpreted as a spectral pattern projected onto a receiver, similar to placing a screen behind a physical prism. The CUDA-SPT, by contrast, starts with camera rays. When the camera looks through the prism toward an emitter, the renderer can show several wavelength-separated apparent images of that emitter. They are virtual images because the renderer infers colored source positions from refracted viewing directions; they are not irradiance patterns deposited on a physical screen.

As illustrated in Fig. 6, the two image types can show an apparently reversed order of colors. In the camera view, the refracted source images are effectively reconstructed from the observer's direction, whereas the forward renderer records where the outgoing rays actually hit the receiver. The apparent reversal therefore follows from comparing a camera view with a projected pattern and does not indicate an incorrect wavelength-to-refractive-index mapping.

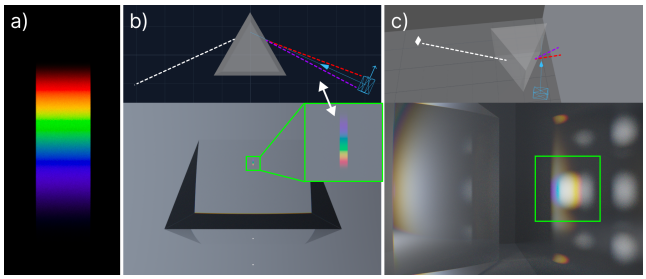


Figure 6: *Comparison between (a) a projected spectrum in the 2D-SLT, (b) a camera-based view through the prism, and (c) a projected dispersion pattern on a metal screen, both rendered by the CUDA-SPT. Viewing the light source through the prism can make the color order appear inverted because the image shows refracted virtual images of the light source instead of irradiance on a receiving screen.*

Projected Caustics The different transport directions also affect how easily the renderers produce a rainbow on a wall. Such a pattern is a specular caustic: a concentrated light pattern created by reflection or refraction. To contribute to the camera image, light must leave the source, traverse the prism at suitable angles, reach the wall, and scatter toward the camera. The forward 2D-SLT samples the first parts of this path directly from the light source. A camera-based path tracer must discover the complete source–prism–wall–camera connection while tracing from the sensor, which is unlikely without specialized caustic methods. For the tested setup, this leads to slow convergence and strong noise in the projected wall pattern, as discussed in Sec. 4.

3.2 Laser Speckle

This subsection describes the physical model and both propagation algorithms of the standalone headless C++ application used for development and evaluation. The browser implementation retains only the FFT-based path and is described in Sec. 3.3.

Laser speckle arises when coherent light is scattered by an optically rough surface and contributions with different optical path lengths interfere at the sensor [Goo07]. Here, coherence means retaining stable phase relationships between scattered contributions. A complex field stores amplitude in its magnitude and phase in its argument. Let $U_n(\mathbf{x})$ denote the n-th complex field contribution at sensor position $\mathbf{x}$. The total field is the coherent sum of these contributions, and the measured intensity is the squared modulus of that total field:

$$U(\mathbf{x}) = \sum_n U_n(\mathbf{x}), \qquad I(\mathbf{x}) = |U(\mathbf{x})|^2. \tag{15}$$

Depending on their relative phases, the contributions interfere constructively or destructively, producing the characteristic granular speckle pattern [Goo07, Eqs. (2.1) and (3.23)].

In contrast to the dispersive prism case discussed in Sec. 3.1, reproducing speckle requires phase to be retained throughout propagation. A purely ray-based model can predict where light reaches the sensor, but not the interference pattern. For this reason, the headless C++ laser-speckle application represents light using complex optical fields.

3.2.1 Scene and Field Representation

The headless C++ application models a coherent optical setup consisting of a laser-illuminated diffuse wall, a thin lens, and a sensor plane. The wall acts as a rough scattering surface; the scattered field is propagated to the lens, modified by the lens transmission, and then propagated to the sensor.

Each optical plane is sampled on a regular two-dimensional grid embedded in three-dimensional space. Rather than storing the full optical field directly, the renderer represents the field on each plane relative to an associated reference wave. The full complex field is written as

$$U(\mathbf{x}) = E(\mathbf{x}) e^{\mathrm{i}kL_{\mathrm{ref}}(\mathbf{x})}, \qquad k = \frac{2\pi}{\lambda}, \\ L_{\mathrm{ref}}(\mathbf{x}) = \hat{\mathbf{d}}_{\mathrm{ref}} \cdot (\mathbf{x} - \mathbf{x}_0), \tag{16}$$

where $U(\mathbf{x})$ denotes the full complex field at position $\mathbf{x}$, $E(\mathbf{x})$ is the complex envelope, $\hat{\mathbf{d}}_{\mathrm{ref}}$ is the unit propagation direction of the reference wave, and $\mathbf{x}_0$ is its reference origin. This representation factors out the rapidly oscillating carrier phase while retaining the amplitude and phase information required for propagation. The envelope varies more slowly across the sampled plane than the full optical field and is therefore easier to represent numerically on a finite grid. The envelope is obtained by removing the reference phase:

$$E(\mathbf{x}) = U(\mathbf{x}) e^{-\mathrm{i}kL_{\mathrm{ref}}(\mathbf{x})}. \tag{17}$$

This representation is used consistently for the wall, lens, and sensor planes.

3.2.2 Illumination and Diffuse Wall Model

To generate a speckle field, the wall must act as a rough scattering surface under coherent illumination [Goo07]. The incident laser spot is modeled by a Gaussian field-amplitude profile [STBE20, pp. 57–59], so that the envelope on the wall is

$$E_{\mathrm{wall}}(\mathbf{x}) = A_0 \exp\left(-\frac{r^2}{w^2}\right) \sqrt{\alpha}\, \rho_{\mathrm{diff}}(\mathbf{x}), \tag{18}$$

where A_0 is the peak field amplitude, r is the distance of $\mathbf{x}$ from the beam axis on the wall, w controls the spot width, and α denotes the diffuse albedo of the wall. The Gaussian term describes the spatial amplitude falloff of the incident beam.

The factor $\rho_{\mathrm{diff}}(\mathbf{x})$ models microscopic surface roughness under coherent illumination. It is represented as a correlated complex random field with unit mean power. Its complex values introduce local amplitude and phase variations, while its correlation length controls the spatial scale of the modeled roughness by making neighboring samples statistically dependent. After propagation, the interference of these locally varying complex contributions produces the observed speckle pattern [Goo07].

3.2.3 Thin-Lens and Sensor Model

After propagation from the wall, the field reaches the lens plane. In the thin-lens approximation, the lens is modeled as a multiplicative phase transformation with a finite circular aperture [Goo05, Eq. (5-10)]:

$$E_{\mathrm{out}}(x,y) = E_{\mathrm{in}}(x,y)\, H_{\mathrm{lens}}(x,y), \tag{19}$$

where $E_{\mathrm{in}}(x,y)$ and $E_{\mathrm{out}}(x,y)$ denote the complex envelopes immediately before and after the lens, respectively. Using local lens-plane coordinates (x,y) measured from the lens center, the transfer function is

$$H_{\mathrm{lens}}(x,y) = \begin{cases} \tau_{\mathrm{lens}} \exp\left(-\mathrm{i}\frac{k}{2f}(x^2+y^2)\right), & x^2+y^2 \le R^2, \\ 0, & \text{otherwise}, \end{cases} \tag{20}$$

where τ_{lens} is the scalar lens-transmission factor, f is the focal length, and R is the aperture radius. The quadratic phase term models the focusing action of the lens, while the aperture suppresses contributions outside the physical lens extent.

At the sensor, the measurable quantity is the intensity rather than the complex field itself [Goo05, Eq. (4-5)]. Since the full field and the envelope differ only by a reference phase factor of unit magnitude, the recorded intensity can be evaluated directly from the propagated envelope as

$$I(\mathbf{x}) = |E(\mathbf{x})|^2. \tag{21}$$

To reduce detector discretization artifacts, the sensor field is evaluated on a supersampled grid and then downsampled to the final output resolution by box averaging.

3.2.4 Wave-Optical Propagation

The headless C++ application propagates a complex optical field from the wall through the lens to the sensor. It provides two schemes: a direct spatial-domain integrator based on the first Rayleigh–Sommerfeld solution and an FFT-based angular-spectrum propagator [Goo05, p. 61]. Both use the envelope-field representation introduced in Sec. 3.2.1, but differ in how free-space propagation is evaluated.

The direct propagator obtains the destination envelope by discretizing the Rayleigh–Sommerfeld diffraction integral as a Riemann sum [Goo05, Eq. (3-51)]; see also [SW06]:

$$E_{\mathrm{dst}}(\mathbf{x}_j) \approx \sum_i E_{\mathrm{src}}(\mathbf{x}_i) \left(-\frac{\mathrm{i}}{\lambda}\right) \frac{\cos\theta_{ij}}{r_{ij}} \\ \cdot \exp\Big(\mathrm{i}k\big[r_{ij} + L_{\mathrm{ref,src}}(\mathbf{x}_i) - L_{\mathrm{ref,dst}}(\mathbf{x}_j)\big]\Big)\, \Delta A, \tag{22}$$

where r_{ij} is the distance between source sample $\mathbf{x}_i$ and destination sample $\mathbf{x}_j$, θ_{ij} is the angle between the source-plane normal and the propagation direction, and ΔA is the source sample area. The two reference-path terms arise because the propagated quantity is the envelope rather than the full field. The sum evaluates the amplitude and phase contribution from every source sample to every destination sample, which explains the point-to-point cost of $\mathcal{O}(N_{\mathrm{src}} N_{\mathrm{dst}})$.

For the FFT-based propagator, the stored envelope is first converted back to the full field on the source plane using Eq. 16. The angular-spectrum method uses a two-dimensional Fourier transform to decompose the source field into plane-wave components represented by transverse spatial frequencies k_x and k_y. Free-space propagation changes each component's phase independently, and an inverse transform reconstructs the propagated field. The full field is decomposed into its angular spectrum,

$$A(k_x,k_y) = \mathcal{F}\{U_{\mathrm{src}}(x,y)\}, \qquad A_z(k_x,k_y) = A(k_x,k_y)e^{\mathrm{i}k_z z}, \quad (23)$$

with

$$k_z = \sqrt{k^2 - k_x^2 - k_y^2}, \qquad k = \frac{2\pi}{\lambda}, \quad (24)$$

where z denotes the signed propagation distance along the source-plane normal. The propagated full field is then reconstructed by

$$U_z(x,y) = \mathcal{F}^{-1}\{A_z(k_x,k_y)\}. \quad (25)$$

Components with $k_x^2 + k_y^2 > k^2$ correspond to non-propagating evanescent components and are discarded in the implementation. Finally, the propagated field is sampled on the destination grid and converted back to envelope form by removing the destination reference phase.

Both propagators use the same optical pipeline but evaluate free-space propagation differently: the direct scheme explicitly sums contributions between source and destination samples, whereas the FFT-based scheme propagates an axial distance z in the spectral domain and samples the result on the destination plane. This reduces the propagation step to approximately $\mathcal{O}(N \log N)$ for an N-sample grid.

3.2.5 Numerical Considerations

Both propagation schemes are sensitive to spatial sampling and finite grid resolution. For the direct Rayleigh–Sommerfeld integrator, the diffraction integral is evaluated as a discrete sum, so insufficient source-plane sampling can reduce accuracy while the point-to-point accumulation quickly becomes expensive [SW06]. To keep this formulation tractable, source samples with negligible field energy are skipped, reference-path terms that are reused across many evaluations are precomputed, and, for wall-to-lens propagation, samples outside the circular lens aperture are excluded by a binary mask.

For the FFT-based propagator, the numerical behavior is determined by the finite computational window and the sampling of the angular spectrum [SW06]. The source field is therefore zero-padded before the forward transform, evanescent components with $k_x^2 + k_y^2 > k^2$ are discarded, and, when finer detector sampling is required, the propagated spectrum is padded before the inverse transform. The reconstructed field is then resampled onto the destination grid by bilinear interpolation.

At the sensor, a coarse detector grid may undersample high-frequency speckle fluctuations. To reduce such artifacts, the sensor intensity is first evaluated on a supersampled grid and then downsampled to the final output resolution by box averaging. These measures improve numerical robustness while keeping the computation tractable and leaving the underlying optical model unchanged.

3.3 Application Architecture

The Vite-based interactive application exposes three components: the 2D-SLT, the CUDA-SPT, and a browser-side FFT-based laser-speckle visualization. The standalone headless C++ application is separate and not connected to the frontend. Because the user-facing components differ in scene representation, preview behavior, dependencies, and execution environment, the application separates renderer-specific functionality behind a common frontend instead of imposing a single renderer interface on all implementations.

The 2D-SLT and the CUDA-SPT run as independent backend services. Each service combines the renderer application with an HTTP adapter, isolating its native dependencies and execution environment from the frontend. This service-oriented design isolates independently deployable functionality [New21]. The browser-side laser-speckle component instead executes the reimplemented FFT path directly through WebGPU and therefore requires no backend service.

On the client side, each renderer is represented by a dedicated strategy that encapsulates its preview logic, validation, and, where required, communication with the corresponding backend service. This application of the Strategy pattern keeps the shared user interface independent of renderer-specific behavior [GHJV09]. As shown in Fig. 7, the 2D-SLT and CUDA-SPT strategies communicate with their renderer services, whereas the laser-speckle strategy invokes the local WebGPU implementation. The headless C++ application remains outside this architecture. All interactive scene previews are implemented with Three.js [mrd26].

3.3.1 Prism Renderers

The 2D-SLT preview displays the configurable scene geometry and approximates dispersion with six fixed wavelength–refractive-index pairs that are refracted independently at prism interfaces. Reflection and absorption of rays are not considered in the preview. The CUDA-SPT preview instead visualizes scene layout and simplified dielectric, emissive, and albedo-based materials. Representative light paths are deliberately omitted because, unlike the parallel rays emitted by the area light in the 2D-SLT, camera-generated paths depend strongly on scene interactions and are therefore less predictable. Both previews are configuration aids rather than approximations of the final rendered radiance (Fig. 1).

3.3.2 Browser-Based Laser-Speckle Component

For frontend integration, the FFT-based laser-speckle path was reimplemented in WebGPU to execute directly on the browser GPU without a backend. This implementation follows the optical model and FFT path of the standalone C++ application but replaces FFTW with a two-dimensional radix-2 FFT implemented in multiple compute passes. It neither includes the direct Rayleigh–Sommerfeld path nor communicates with the headless application. In the preview, the wall and lens planes display the simulated field-intensity data rather than their directly visible optical appearance. The browser-based component is shown in Fig. 1.

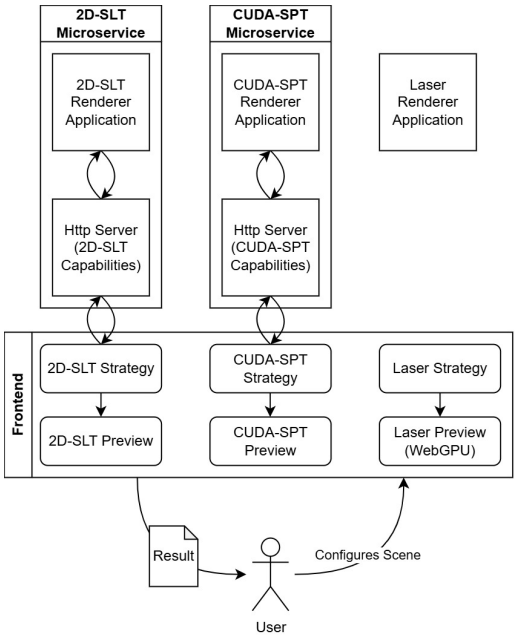


Figure 7: *PROP application architecture. Renderer-specific frontend strategies provide previews and invoke either independent backend services or the local WebGPU implementation. The standalone headless C++ laser-speckle application used for algorithm development and evaluation is separate and is not shown.*

4 Evaluation

The prism renderers and headless C++ laser-speckle application are evaluated separately because they target different phenomena, use different propagation models, and were benchmarked in different implementation environments. For the prism renderers, the evaluation focuses on runtime scaling and the qualitative interpretation of their different observables. For laser speckle, the two propagation backends of the headless C++ application are compared by runtime and numerical agreement; the Vite/WebGPU component is not benchmarked. All experiments were conducted on an AMD Ryzen AI 7 350 processor with eight physical cores, 16 logical threads, and 33.6 GB of system memory. The CUDA experiments used an NVIDIA RTX 5070 Ti laptop GPU with 12.0 GB of dedicated memory.

4.1 Prism Renderers

The prism evaluation measures runtime scaling with each renderer's principal configuration parameters. Because the renderers produce different observables, their outputs are interpreted qualitatively rather than treated as estimates of the same image quantity.

4.1.1 Setup and Metrics

We created matched scene layouts for both renderers, each comprising a prism with a 60° apex angle made of BK7 optical glass, a small area light positioned at the bottom left using the D65 SPD, and a camera or image plane located at the bottom right. Baseline render configurations were chosen individually for each renderer and are summarized in Tab. 1. Each configuration was run five times after one warm-up run. We report the mean runtime, including PPM output, but excluding scene, light, and material preprocessing. The baseline runtimes are not directly comparable because the CUDA-SPT traces 64 times more rays than the 2D-SLT in every configuration. The measurements are therefore used only to analyze scaling within each renderer.

4.1.2 Results

Tab. 1 reports runtime scaling for each renderer. For the 2D-SLT, the observed scaling depends strongly on the affected rendering stage. Increasing the number of light samples only affects recursive light tracing, which is executed on the GPU using CUDA. Consequently, even an eightfold increase in light samples results in a runtime increase of only $1.15\times$. In contrast, increasing the spectral resolution affects the spectral-to-RGB conversion performed on the CPU, leading to a runtime increase of $3.01\times$ for an eightfold increase in spectral samples. The image resolution also affects this stage. Since both the image width and height were scaled simultaneously, the total number of pixels grows quadratically with the specified multiplier. As a result, increasing the image dimensions by a factor of eight increases runtime by a factor of 42.89. Although it runs entirely on the GPU, the CUDA-SPT runtime more closely tracks the parameter multipliers. Increasing samples per pixel or spectral samples has a similar effect because both parameters increase the number of traced rays. As for the 2D-SLT, scaling both image dimensions increases the number of traced samples, and thus runtime, approximately quadratically.

4.1.3 Discussion

The prism measurements characterize scaling within each renderer and should not be interpreted as a direct performance ranking. The renderers trace different path families, use independently selected baseline configurations, and produce different observables. For the 2D-SLT, the weak runtime increase with additional light samples reflects GPU-parallel ray tracing, whereas the stronger dependence on spectral and image resolution identifies the CPU-based spectral-to-RGB conversion as the dominant bottleneck for those parameters. The CUDA-based spectral path tracer is more general, supporting three-dimensional scenes, camera views, indirect light transport, and several material types. However, this flexibility comes at the cost of slower convergence for the projected prism spectrum considered here. The spectrum on a diffuse receiver is a specular caustic: the relevant paths must travel from the source through the prism to the receiver and then to the camera. Ordinary backward path tracing samples these paths only rarely, which leads to noise and makes the projected spectrum difficult to render reliably.

The 2D-SLT has a complementary scope. It directly simulates forward light transport through the prism and records where different wavelengths arrive on a projection plane. This makes it suitable for analyzing projected dispersion patterns, but it cannot render the full three-dimensional appearance of a scene or a view through the prism from an arbitrary camera position. A combined workflow could therefore use the 2D-SLT to compute projected spectral

Table 1: *Mean runtimes over five measured runs for each benchmark multiplier. The smaller value below each runtime is relative to the corresponding baseline. Values should be compared only within a renderer because the baseline configurations and traced path counts are not equivalent.*

Renderer	Parameter	Baseline value	×0.25	×0.5	Baseline	×2	×4	×8
2D-SLT	Light samples	2^{24}	309.60 ms 1.02×	310.60 ms 1.03×	302.20 ms 1.00×	317.40 ms 1.05×	330.40 ms 1.09×	348.60 ms 1.15×
	Spectral resolution	64	257.60 ms 0.85×	279.00 ms 0.92×	302.20 ms 1.00×	389.00 ms 1.29×	519.40 ms 1.72×	909.40 ms 3.01×
	Image width and height	512	102.20 ms 0.34×	154.80 ms 0.51×	302.20 ms 1.00×	969.60 ms 3.21×	3937.20 ms 13.03×	12961.40 ms 42.89×
CUDA-SPT	Samples per pixel	64	304.94 ms 0.42×	457.15 ms 0.64×	718.50 ms 1.00×	1396.18 ms 1.94×	1981.63 ms 2.76×	3839.26 ms 5.34×
	Spectral samples	64	253.40 ms 0.35×	362.58 ms 0.50×	718.50 ms 1.00×	1088.76 ms 1.52×	1996.95 ms 2.78×	3858.14 ms 5.37×
	Image width and height	512	103.60 ms 0.14×	157.31 ms 0.22×	718.50 ms 1.00×	2324.35 ms 3.24×	9257.57 ms 12.88×	36812.64 ms 51.24×

distributions and the CUDA path tracer for the camera-dependent appearance of the prism itself.

4.2 Headless C++ Laser-Speckle Application

The laser-speckle evaluation uses only the standalone headless C++ application that formed the basis for the propagation algorithms and experiments. It compares the FFT-based angular-spectrum propagator with the direct Rayleigh–Sommerfeld integrator; the Vite/WebGPU implementation is excluded.

4.2.1 Setup and Metrics

Both variants share the same code to isolate the propagation scheme. The benchmark paths differ only in how they evaluate free-space propagation from wall to lens and lens to sensor.

The benchmark consists of 14 deterministic cases formed by combining two laser presets with seven sensor presets. All runs use the same random seed, optical parameters, sampling configuration, and a zero-padding factor of 4 for FFT-based propagation. The detector-image width was fixed at 300 pixels. The evaluated presets are summarized in Tab. 2.

For aggregate analysis, sensor presets S1, S2, S3, and S6 form the *parallel-like* group because the sensor remains parallel to the lens plane; S4, S5, and S7 form the *tilted-like* group. Each configuration is evaluated once with the direct propagator and once with the FFT-based propagator. We report single-run timings because each direct-reference case requires several minutes.

Numerical agreement is evaluated at three stages of the pipeline: after wall-to-lens propagation, on the supersampled sensor field, and on the final downsampled detector image. At the lens and sensor stages, we compare intensities derived from the propagated envelopes, i.e., $|E|^2$, rather than the complex envelopes themselves. For a reference value r_p from the direct integrator and a corresponding value c_p from the FFT-based method, the relative RMSE

Table 2: *Laser (L) and sensor (S) presets combined to form the 14 deterministic speckle benchmark cases.*

Type	Preset	Description
Laser	L1	The beam is centered on the wall and propagates orthogonally to it.
Laser	L2	45° oblique incidence in the x–z plane while illuminating the same wall region.
Sensor	S1	The sensor plane remains parallel and centered to the lens plane. Autofocus enabled.
Sensor	S2	S1 with a lateral shift in x.
Sensor	S3	S1 with a diagonal lateral shift.
Sensor	S4	S1 with a strong tilt.
Sensor	S5	S1 with a strong tilt and lateral shift.
Sensor	S6	S1 without autofocus (defocused).
Sensor	S7	S1 with a diagonal grazing-angle pose.

is defined as

$$\mathrm{RMSE}_{\mathrm{rel}}(r,c) = \sqrt{\frac{\sum_p (c_p - r_p)^2}{\sum_p r_p^2}}. \tag{26}$$

For the final detector image, we additionally report the Pearson correlation coefficient to quantify structural similarity.

4.2.2 Results

Tab. 3 summarizes the aggregated benchmark results. Across all 14 cases, the FFT-based propagator reduced mean runtime from 302.5 s to 0.991 s, an average speedup of 305.2×. The mean detector-image correlation was 0.929, and the mean relative RMSE was 0.335. Fig. 8 compares representative detector images. Under normal incidence (L1/S1), both propagators produce nearly identical speckle patterns. Agreement decreases under oblique illumination (L2/S1) and further with a grazing-angle sensor pose (L2/S7).

The dominant trend is the difference between normal (L1) and

Table 3: *Aggregated laser-speckle benchmark results. Runtime columns report group means in seconds; each individual configuration was executed once per propagator. RMSE and correlation refer to the final detector images.*

Group	Direct (s)	FFT (s)	Speedup	$\mathrm{RMSE}_{\mathrm{rel}}$	Corr.
All cases	302.5	0.991	305.2×	0.335	0.929
L1	295.2	0.990	298.1×	0.181	0.982
L2	309.8	0.992	312.4×	0.489	0.875
Parallel-like	308.7	0.991	311.4×	0.315	0.940
Tilted-like	294.1	0.990	297.0×	0.362	0.914

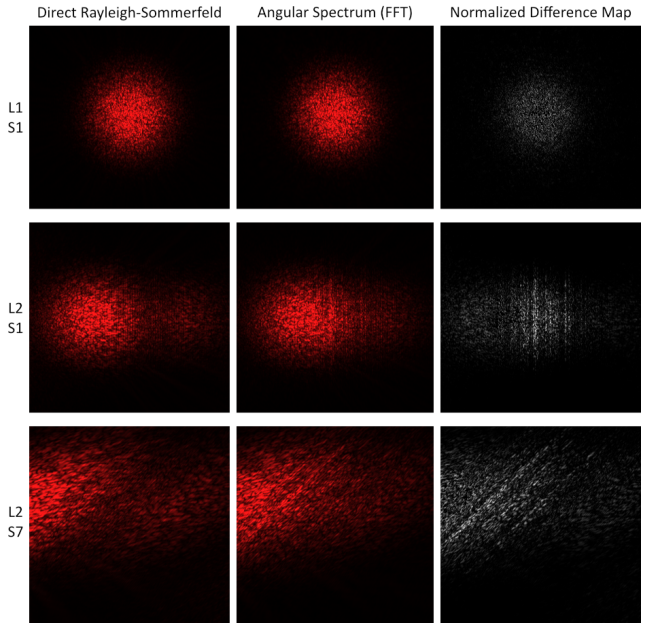


Figure 8: *Comparison of the direct Rayleigh–Sommerfeld and FFT-based angular-spectrum propagators for three representative laser–sensor configurations. The first row shows normal incidence with a parallel sensor (L1/S1), the second row shows* 45° *oblique incidence with a parallel sensor (L2/S1), and the third row combines oblique incidence with a grazing-angle sensor pose (L2/S7). The difference maps visualize the spatial distribution of the pixel-wise RGB error. Agreement is highest for normal incidence and decreases for oblique illumination and strongly tilted sensor configurations.*

oblique (L2) illumination. Under normal incidence, the FFT-based method agrees well with the direct reference: the lens-plane relative RMSE is constant at 0.077 across all sensor presets, the mean image RMSE is 0.181, and the mean image correlation reaches 0.982. Under 45° oblique illumination, agreement is already lower at the lens plane, where relative RMSE rises to 0.463 for all sensor presets. The discrepancy persists through the optical pipeline, yielding a mean image RMSE of 0.489 and correlation of 0.875.

The sensor presets show a second, smaller trend. Parallel-like sensor configurations achieve a mean image correlation of 0.940 and a mean image RMSE of 0.315, whereas tilted-like configurations reach 0.914 and 0.362, respectively. The difference between the parallel-like and tilted-like groups is smaller than the difference between normal and oblique illumination, although the disagreement is most visible in preset S7.

4.2.3 Discussion

The FFT-based propagator is most effective when runtime is the dominant concern and propagation remains predominantly aligned with the normals of the sampled optical planes. In these cases, it remains numerically close to the direct reference at a small fraction of the runtime. The main difference between L1 and L2 is already visible at the lens plane and therefore cannot be attributed primarily to sensor tilt.

This behavior is consistent with the field representation. In the direct propagator, the reference-wave phase is incorporated analytically in the Rayleigh–Sommerfeld sum, whereas the FFT-based path first reconstructs the full field $U(\mathbf{x}) = E(\mathbf{x})e^{\mathrm{i}kL_{\mathrm{ref}}(\mathbf{x})}$ on a finite Cartesian grid. For oblique illumination, reconstructing the full field introduces a steep phase ramp across the finite Cartesian grid; this off-axis carrier is represented less robustly at the current sampling density [SW06]. Strongly tilted sensor poses add a second error source because the current implementation resamples an axially propagated field onto the detector plane. The results therefore indicate that the current FFT implementation is most reliable for configurations close to axial propagation, while agreement decreases for increasingly off-axis geometries.

5 Limitations and Future Work

The prism evaluation compares runtime scaling within each renderer, but it does not establish a direct accuracy or performance ranking between them because the 2D-SLT and CUDA-SPT produce different observables and use different sampling configurations. For the CUDA path tracer, future work should investigate caustic-oriented techniques such as bidirectional path tracing, photon mapping, path guiding, or explicit forward-ray sampling from the light source. Acceleration structures, denoising, and more targeted spectral sampling could further reduce runtime in more complex scenes.

Moving the 2D-SLT spectral-to-RGB conversion to the GPU could reduce the runtime dependence on spectral and image resolution. The renderer could also be extended into a more versatile visualization tool supporting additional materials, SPDs, and geometries. To this end, it should adhere more closely to physically meaningful measurements and units and incorporate diffuse scattering from imperfect surfaces.

For laser speckle, off-axis or shifted angular-spectrum formulations, beam-aligned reference-wave formulations, and tilted-plane or non-uniform FFT methods are promising extensions for oblique illumination and tilted detector planes [MSW03, XTQ*16]. The current evaluation compares two numerical implementations of the same scene model; assessing physical accuracy would require validation against measured surface statistics and real speckle images.

6 Conclusion

This work examined which properties of light must be retained to model prism dispersion and laser speckle and the trade-offs of the corresponding problem-specific renderers. Within PROP's smooth-dielectric model, prism dispersion requires wavelength-resolved geometric transport. The 2D-SLT traces spectral contributions forward onto a receiver, whereas the integrated CUDA-SPT produces camera-based spectral images. Because they expose different observables and use different sampling configurations, the evaluation does not rank their accuracy or performance directly. Instead, it shows that the 2D-SLT scales efficiently with emitted light samples but is increasingly limited by its CPU-based spectral-to-RGB conversion, while the CUDA-SPT runtime more closely follows the number of traced spectral samples.

Laser speckle additionally requires phase-preserving wave propagation because its detector pattern results from coherent interference. In the evaluated headless C++ pipeline, FFT-based angular-spectrum propagation reduced mean runtime from 302.5 s to 0.991 s, a speedup of 305.2×, while achieving a mean detector-image correlation of 0.929 with the direct Rayleigh–Sommerfeld integrator. Agreement was highest under normal incidence and decreased for increasingly off-axis illumination and tilted detector configurations.

Overall, the light representation must match the target phenomenon: wavelength-resolved ray transport captures prism dispersion, whereas laser speckle also requires amplitude and phase. Problem-specific restrictions focus computation on the required transport quantities, but efficiency, the rendered observable, and numerical agreement remain tied to explicit model and geometry assumptions.

References

[Bal08] BALL D. W.: Prisms | Spectroscopy Online, Sept. 2008. URL: https://www.spectroscopyonline.com/view/prisms.

[Bau26] BAUDOIN T.: CUDA-Spectral-Pathtracer, June 2026. original-date: 2025-03-11T02:45:20Z. URL: https://github.com/TheoBaudoinLighting/CUDA-Spectral-Pathtracer.

[DLW93] DUTRÉ P., LAFORTUNE E. P., WILLEMS Y. D.: Monte Carlo light tracing with direct computation of pixel intensities. *3rd International Conference on Computational Graphics and Visualisation Techniques* (Dec. 1993), 128–137. URL: http://graphics.cs.kuleuven.be/publications/MCLTWDCOPI/MCLTWDCOPI_paper.ps.

[DM98] DAGUM L., MENON R.: OpenMP: an industry standard API for shared-memory programming. *IEEE Computational Science and Engineering 5*, 1 (Jan. 1998), 46–55. URL: https://ieeexplore.ieee.org/document/660313, doi:10.1109/99.660313.

[EM99] EVANS G. F., MCCOOL M. D.: Stratified Wavelength Clusters for Efficient Spectral Monte Carlo Rendering. In *Proceedings of the Graphics Interface 1999 Conference, June 2-4, 1999, Kingston, Ontario, Canada* (June 1999), pp. 42–49. URL: http://graphicsinterface.org/wp-content/uploads/gi1999-7.pdf.

[GHJV09] GAMMA E., HELM R., JOHNSON R., VLISSIDES J.: *Entwurfsmuster : Elemente wiederverwendbarer objektorientierter Software*. Addison Wesley, München, 2009.

[Goo05] GOODMAN J. W.: *Introduction to Fourier Optics*. Roberts and Company Publishers, 2005. Google-Books-ID: ow5xs_Rtt9AC.

[Goo07] GOODMAN J. W.: *Speckle Phenomena in Optics: Theory and Applications*. Roberts and Company Publishers, 2007. Google-Books-ID: TynXEcS0DncC.

[Hec16] HECHT E.: *Optics*. Pearson International, 2016. URL: https://elibrary.pearson.de/book/99.150005/9781292096964?lang=en.

[IEC99] IEC: *IEC 61966-2-1:1999: Multimedia Systems and Equipment – Colour Measurement and Management – Part 2-1: Colour Management – Default RGB Colour Space – sRGB*. Tech. Rep. IEC 61966-2-1:1999, International Electrotechnical Commission, Geneva, Switzerland, 1999.

[IWR*14] IKEDA S., WATANABE S., RAYTCHEV B., TAMAKI T., KANEDA K.: Spectral Rendering of Interference Phenomena Caused by Multilayer Films Under Global Illumination Envi- ronment. *ITE Transactions on Media Technology and Applications 3*, 1 (2014), 76–84. URL: https://api.semanticscholar.org/CorpusID:207784062.

[KDR18] KRUISSELBRINK T., DANGOL R., ROSEMANN A.: Photometric measurements of lighting quality: An overview. *Building and Environment 138* (June 2018), 42–52. URL: https://linkinghub.elsevier.com/retrieve/pii/S0360132318302397, doi:10.1016/j.buildenv.2018.04.028.

[KG24] KIM A., GOTCHEV A.: Spectral Rendering: From Input to Rendering Process. *Electronic Imaging 36* (Jan. 2024), 1–6. URL: https://library.imaging.org/ei/articles/36/10/IPAS-251, doi:10.2352/EI.2024.36.10.IPAS-251.

[Maj21] MAJERCIK Z.: The Schlick Fresnel Approximation. In *The Schlick Fresnel Approximation*. Apress, Berkeley, CA, Aug. 2021, pp. 109–114. doi:10.1007/978-1-4842-7185-8_9.

[MPP20] MAYERHÖFER T. G., PAHLOW S., POPP J.: The Bouguer-Beer-Lambert Law: Shining Light on the Obscure. *Chemphyschem 21*, 18 (Sept. 2020), 2029–2046. URL: https://pmc.ncbi.nlm.nih.gov/articles/PMC7540309/, doi:10.1002/cphc.202000464.

[mrd26] MRDOOB: mrdoob/three.js, July 2026. original-date: 2010-03-23T18:58:01Z. URL: https://github.com/mrdoob/three.js.

[MSW03] MATSUSHIMA K., SCHIMMEL H., WYROWSKI F.: Fast calculation method for optical diffraction on tilted planes by use of the angular spectrum of plane waves. *JOSA A 20*, 9 (Sept. 2003), 1755–1762. URL: https://opg.optica.org/josaa/abstract.cfm?uri=josaa-20-9-1755, doi:10.1364/JOSAA.20.001755.

[New21] NEWMAN S.: *Building Microservices: Designing Fine-Grained Systems*. "O'Reilly Media, Inc.", July 2021. Google-Books-ID: ZvM5EAAAQBAJ.

[noa] Datasets | CIE. URL: https://www.cie.co.at/data-tables.

[Pas13] PASCHOTTA D. R.: Fresnel Equations. *RP Photonics Encyclopedia* (Apr. 2013). URL: https://www.rp-photonics.com/fresnel_equations.html, doi:10.61835/tql.

[PJH23] PHARR M., JAKOB W., HUMPHREYS G.: *Physically Based Rendering, fourth edition: From Theory to Implementation*. The MIT Press, Cambridge, Massachusetts London, England, 2023.

[RBA09] RADZISZEWSKI M., BORYCZKO K., ALDA W.: An Improved Technique for Full Spectral Rendering. *Journal of WSCG 17* (Jan. 2009).

[SCH21] SCHOTT: Optisches Glas Datenblattsammlung, Sept. 2021. URL: https://www.schott.com/de-de/products/optical-glass-p1000267/downloads.

[SP25] STEINBERG S., PHARR M.: Wave Tracing: Generalizing The Path Integral To Wave Optics, Aug. 2025. URL: https://arxiv.org/abs/2508.17386v1.

[SSY22] STEINBERG S., SEN P., YAN L.-Q.: Towards practical physical-optics rendering. *ACM Transactions on Graphics (TOG) 41*, 4 (July 2022), 132:1–132:24. URL: https://dl.acm.org/doi/10.1145/3528223.3530119, doi:10.1145/3528223.3530119.

[STBE20] SALEH B. E. A., TEICH M. C., BAR-ELI M.: *Optik und Photonik*. John Wiley & Sons, Incorporated, Newark, GERMANY, 2020. URL: http://ebookcentral.proquest.com/lib/hsrm/detail.action?docID=6184688.

[SW06] SHEN F. B., WANG A.: Fast-Fourier-Transform Based Numerical Integration Method For The Rayleigh-Sommerfeld Diffraction Formula. *Applied Optics Vol. 45, No. 6* (Aug. 2006). URL: http://hdl.handle.net/10919/46900.

[SY21] STEINBERG S., YAN L.-Q.: A generic framework for physical light transport. *ACM Transactions on Graphics 40*, 4 (Aug. 2021), 1–20. URL: https://dl.acm.org/doi/10.1145/3450626.3459791, doi:10.1145/3450626.3459791.

[SY22] STEINBERG S., YAN L.-Q.: Rendering of Subjective Speckle Formed by Rough Statistical Surfaces. *ACM Transactions on Graphics (TOG) 41*, 1 (Feb. 2022), 2:1–2:23. URL: https://dl.acm.org/doi/10.1145/3472293, doi:10.1145/3472293.

[WND*14] WILKIE A., NAWAZ S., DROSKE M., WEIDLICH A., HANIKA J.: Hero Wavelength Spectral Sampling. *Computer Graphics Forum 33*, 4 (July 2014), 123–131. URL: https://onlinelibrary.wiley.com/doi/10.1111/cgf.12419, doi:10.1111/cgf.12419.

[WSND26] WENZEL JAKOB, SÉBASTIEN SPEIERER, NICOLAS ROUSSEL, DELIO VICINI: Mitsuba3, July 2026. original-date: 2021-04-28T13:50:41Z. URL: https://github.com/mitsuba-renderer/mitsuba3.

[WSS13] WYMAN C., SLOAN P.-P., SHIRLEY P.: Simple Analytic Approximations to the CIE XYZ Color Matching Functions. *Journal of Computer Graphics Techniques 2*, 2 (2013).

[XTQ*16] XIAO Y., TANG X., QIN Y., PENG H., WANG W., ZHONG L.: Nonuniform fast Fourier transform method for numerical diffraction simulation on tilted planes. *JOSA A 33*, 10 (Oct. 2016), 2027–2033. URL: https://opg.optica.org/josaa/abstract.cfm?uri=josaa-33-10-2027, doi:10.1364/JOSAA.33.002027.

[Zim23] ZIMMER S. J.: Prism (optics) | Science | Research Starters | EBSCO Research, 2023. URL: https://www.ebsco.com.